

◆ TECHNICAL REQUIREMENTS

KINETIC TYPE

Technical Requirements Document

System architecture, technology stack, and engineering requirements for the adaptive typing engine — an open-source-first stack built on a local-first IndexedDB core, with Firebase as the cloud sync & identity layer and Vercel for hosting & delivery.

REACT + TYPESCRIPT

FIREBASE

VERCEL HOSTING

WEB WORKERS

CANVAS RENDERER

Table of Contents

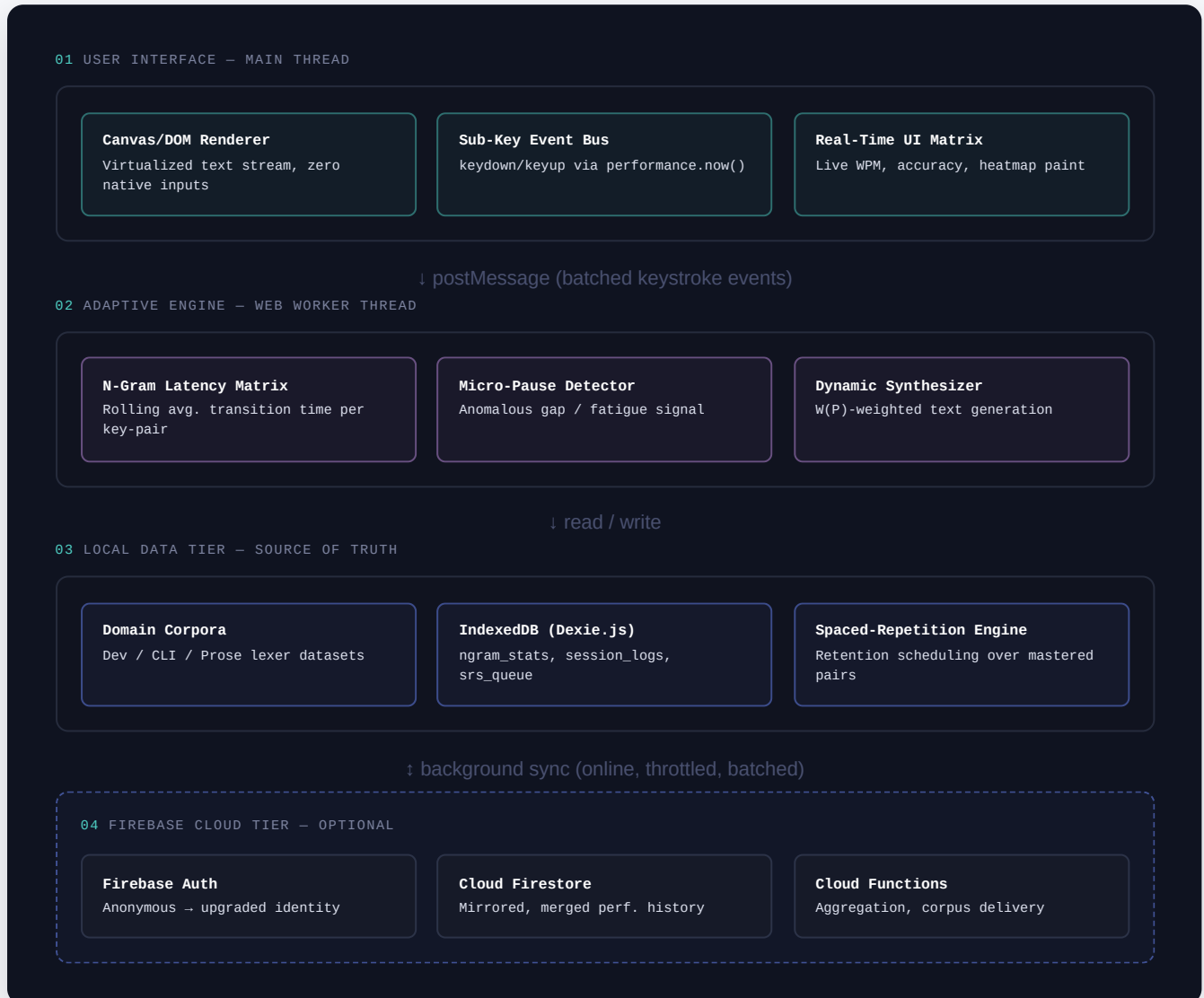
01	Architecture Overview	03	02	Technology Stack	04
03	Core Engine Components	05	04	Latency & Adaptive Weighting Algorithm	06
05	Cloud Architecture — Firebase & Vercel	07	06	Local ⇄ Cloud Sync Strategy	08
07	Interface Contracts	09	08	Performance Budgets	09
09	Security & Privacy	10	10	Testing Strategy	10
11	Environments & CI/CD	11	12	Dependency Manifest	11

Note on backend choice — the source architecture spec used a generic cloud placeholder; this TRD standardizes on **Firebase** (Auth, Firestore, Cloud Functions) in place of Supabase, chosen for tighter client-SDK offline-sync primitives that pair naturally with the product's local-first IndexedDB core.

Note on hosting & stack — static hosting and CDN delivery run on **Vercel** (Git-integrated preview/production deploys), not Firebase Hosting. Every library in §02 is open-source (MIT/Apache-2.0/BSD) or a web standard; the only non-open-source components are the two deliberately chosen managed platforms — Firebase's backend services and Vercel's hosting infrastructure — both flagged explicitly in the License column.

01 Architecture Overview

Kinetic Type runs a decoupled four-tier architecture: an ultra-low-latency presentation tier, a background analytical worker tier, a local-first data tier, and — new in this revision — a cloud sync tier on Firebase that is entirely optional at runtime.



Tiers 01–03 are fully functional with zero network access. Tier 04 activates only after opt-in sign-in and never blocks the typing critical path.

02 Technology Stack

LAYER	TECHNOLOGY	RATIONALE	LICENSE
UI Framework	React 18 + TypeScript	Component isolation for settings/analytics chrome; typing surface itself bypasses React's render cycle entirely (see §03).	MIT
Typing Surface	Custom Canvas 2D renderer	Native form inputs and DOM text nodes introduce composition/layout latency; a virtualized canvas buffer guarantees frame-exact paint timing.	WEB STANDARD
Event Timing	performance.now() + Pointer/Keyboard Events API	Sub-millisecond monotonic clock, immune to system clock adjustments.	WEB STANDARD
Background Compute	Web Worker (dedicated)	N-gram matrix updates, synthesizer text assembly, and anomaly detection run off the main thread so they never compete with input-to-paint latency.	WEB STANDARD
Local Persistence	IndexedDB via Dexie.js	Structured, indexed, promise-based local storage; source of truth per FR-09.	APACHE-2.0
State/Store	Zustand	Minimal-overhead global state for session/UI, decoupled from the worker's own internal state.	MIT
Styling	Tailwind CSS + CSS Modules	Design-system tokens shared with the UI/UX Brief; scoped styles for the canvas overlay chrome.	MIT
Worker Messaging	Comlink	Ergonomic RPC-style wrapper over <code>postMessage</code> , avoids hand-rolled message routing.	APACHE-2.0
Identity	Firebase Authentication	Anonymous auth by default (frictionless first session), upgradeable to Email/Google without losing local history.	MANAGED PLATFORM SDK: Apache-2.0
Cloud Database	Cloud Firestore	Document model maps directly onto the existing IndexedDB collections; native offline persistence & real-time listeners.	MANAGED PLATFORM SDK: Apache-2.0
Serverless Compute	Cloud Functions for Firebase (2nd gen)	Nightly aggregation jobs, corpus versioning endpoint, abuse/rate-limit guarding for writes.	MANAGED PLATFORM SDK: Apache-2.0
Hosting / CDN	Vercel	Git-integrated deploys with automatic PR preview URLs and a global edge network for the static app bundle and corpus JSON payloads.	MANAGED PLATFORM CLI: MIT
Analytics	PostHog	Self-hostable, event-based product analytics for funnel and retention analysis — replaces a vendor-locked analytics pipeline.	MIT
Monitoring	Sentry (self-hosted) + web-vitals	Error tracking and real-user Core Web Vitals, both self-hostable — correlates latency regressions with error spikes without a proprietary APM.	OPEN-SOURCE ¹ / APACHE-2.0
CI/CD	GitHub Actions	Lint → test → build pipeline; deploys hand off to Vercel's Git integration for preview/production promotion.	RUNNER: MIT

¹ Self-hosted Sentry ships under the Functional Source License (source-available, converts to Apache-2.0 after 2 years); the client SDKs used in this app are MIT-licensed. All other rows are OSI-approved open source or public web standards. Firebase and Vercel are the only proprietary managed platforms in the stack — both deliberate architecture choices, not open-source software.

03 Core Engine Components

SUB-KEY EVENT BUS

Every keydown/keyup is timestamped at capture time and pushed onto a ring buffer, flushed to the Adaptive Engine worker in batches of 32 events (or every 50ms, whichever comes first) via `postMessage` with transferable `ArrayBuffer` payloads to avoid structured-clone overhead.

```
interface KeyEvent {
  code: string; // e.g. "KeyE", "ShiftLeft"
  char: string; // resolved printable character
  type: 'down' | 'up';
  t: number; // performance.now() timestamp
  domain: 'CODE_TS' | 'CLI_BASH' | 'PROSE';
}
```

N-GRAM LATENCY MATRIX

The worker maintains an in-memory `Map<pair_id, PairStat>` mirrored to IndexedDB. Transit time for pair $P = (k_1, k_2)$ is computed as $t_{\text{transit}} = t_{\text{keydown}}(k_2) - t_{\text{keyup}}(k_1)$, and folded into an exponentially-weighted moving average with smoothing factor $\lambda = 0.15$ to stay responsive to recent performance without noise sensitivity.

ZERO-LATENCY RENDER ENGINE

The typing stage is a single `<canvas>` element with a virtualized character buffer. Only the glyph run within the visible viewport plus a 2-line lookahead is redrawn per frame, using `requestAnimationFrame` gated so no more than one paint occurs per event batch. This removes: native caret rendering, IME composition delay, and reflow from DOM text node mutation.

ENGINEERING CONSTRAINT

The main thread must never block on IndexedDB writes during an active session. All persistence during a session is queued in memory and flushed asynchronously; only session-boundary events (start/end) await a write confirmation.

04 Latency & Adaptive Weighting Algorithm

Selection weight for a given key-pair in the next text-generation cycle combines its raw speed cost with an error penalty, so pairs that are both slow *and* error-prone are prioritized most aggressively:

$$W(P) = L(P) \times (1 + \alpha \cdot E(P))$$

W(P) – selection weight for key-pair P in the next generation cycle
L(P) – exponentially-weighted moving average of transition latency (ms)
E(P) – historical error rate for P over the last N attempts, $0 \leq E(P) \leq 1$
α – error weight multiplier, default 2.5 (tunable per-domain via remote config)

TEXT SELECTION PROTOCOL



Input: Corpus C, Latency Matrix M, Target Length K
Output: Generated Text Sequence S

1. Retrieve lowest-performing pairs **P_target** from M, ranked by **W(P)** descending.
2. Query domain Corpus C for tokens containing **P_target** matches.
3. Assemble S balancing **70%** targeted high-weight tokens and **30%** contextual flow tokens.
4. Apply Spaced-Repetition gate: pairs mastered < 3 sessions ago are re-inserted at low density to confirm retention.
5. Render Sequence S into the virtualized display buffer.

WORKER EXECUTION BUDGET

OPERATION	TRIGGER	BUDGET
Matrix update (batch of events)	Every 50ms or 32 events	< 4ms
Weight recomputation W(P)	Every 2s, or on session start	< 8ms
Next-chunk synthesis	When buffer < 40 chars remaining	< 6ms
IndexedDB flush	Every 5s, debounced	Non-blocking (async)

05 Cloud Architecture — Firebase & Vercel

FIREBASE AUTHENTICATION

Frictionless → Durable Identity

Every session starts under `signInAnonymously()`. On explicit opt-in, `linkWithCredential()` upgrades to Email/Password or Google OAuth *without changing the UID*, so Firestore history stays attached.

CLOUD FIRESTORE

Mirrors the local schema

Collections map 1:1 to IndexedDB tables (`ngram_stats`, `session_logs`) under `users/{uid}/...`, keeping the sync layer a straight document diff rather than a schema translation. Full field reference: Backend Schema doc.

CLOUD FUNCTIONS (2ND GEN)

Server-side aggregation

Scheduled function `nightlyAggregate` rolls per-pair stats into denormalized leaderboards & percentile bands; an HTTPS callable `getCorpusManifest` serves versioned domain corpora so lexers can update without a client release.

VERCEL

Static bundle + corpus CDN

The React app, canvas engine bundle, and static corpus JSON are served from Vercel's global edge network. Every branch and PR gets an automatic preview deployment; merges to `main` promote straight to production with zero manual deploy steps.

WHY FIREBASE OVER A POSTGRES-BASED BAAS (E.G. SUPABASE)

CONSIDERATION	FIREBASE
Offline-first client SDK	Firestore's SDK ships built-in offline persistence and listener queuing, mirroring the app's own IndexedDB-first philosophy with less bespoke sync code.
Document model fit	Per-user nested collections (<code>ngram_stats</code> as a subcollection) map cleanly onto Firestore's document/subcollection model without join tables.
Anonymous-to-durable auth	Native anonymous auth with in-place account linking avoids a "guest data migration" problem.
Serverless aggregation	Scheduled & callable Cloud Functions cover nightly rollups without provisioning a separate compute tier.

WHY VERCEL FOR HOSTING

CONSIDERATION	VERCEL
Git-native deploys	Every push gets a unique preview URL; no separate deploy command or hosting console step in the day-to-day workflow.
Decoupled from the backend vendor	Hosting is independent of the Firebase project, avoiding a single-vendor lock-in for both compute and delivery.
Edge network performance	Static assets and corpus JSON are served from edge locations close to the user, keeping time-to-first-keystroke low (\$08).

06 Local ⇄ Cloud Sync Strategy

IndexedDB remains the source of truth at all times. Firestore is a mirror used for backup and multi-device continuity — the app must never require a round-trip to Firestore to render or update the typing UI.

SYNC ALGORITHM

1. **Write-through queue:** every local mutation to `ngram_stats` or a completed `session_logs` row is appended to an outbound sync queue table in IndexedDB.
2. **Debounced flush:** when signed in and online, the queue drains in batches (max 20 docs) via `writeBatch()`, throttled to once per 10 seconds.
3. **Conflict resolution — per-pair merge, not last-write-wins:** for `ngram_stats`, the merge function recombines `avg_latency_ms` and `total_occurrences` from both local and remote using a weighted-average by occurrence count, so a second device never silently overwrites the first's history.
4. **Session logs are append-only:** immutable once written; sync is a pure upsert keyed by `session_id` (UUID generated client-side), so duplicate delivery is naturally idempotent.
5. **Cold-start hydration:** a brand-new device signing into an existing account pulls the full `ngram_stats` collection once and seeds a fresh local IndexedDB store before enabling the write-through queue.

DESIGN PRINCIPLE

Sync is additive and best-effort. If Firestore is unreachable for days, the product experience is unaffected; the queue simply grows and drains on next connectivity.

OFFLINE DETECTION

The sync module listens to `navigator.onLine` plus a lightweight Firestore connectivity probe (`.info/connected` equivalent via a snapshot listener on a sentinel doc), since `navigator.onLine` alone under-reports captive-portal and DNS failure states.

07 Interface Contracts

MAIN THREAD ⇄ ADAPTIVE ENGINE WORKER

MESSAGE	DIRECTION	PAYLOAD
KEY_EVENT_BATCH	Main → Worker	<code>KeyEvent[]</code> (transferable buffer)
MATRIX_UPDATED	Worker → Main	Delta of changed <code>pair_id</code> entries, for heatmap repaint
NEXT_CHUNK_READY	Worker → Main	Synthesized character sequence + target pair annotations
SESSION_END	Main → Worker	Session metadata; triggers final aggregation + IndexedDB flush

CLIENT SDK ⇄ FIREBASE

OPERATION	FIREBASE API
Anonymous session start	<code>signInAnonymously(auth)</code>
Account upgrade	<code>linkWithCredential(user, credential)</code>
Queue flush	<code>writeBatch(db).set(doc, data, {merge:true})</code>
Cold-start hydration	<code>getDocs(collection(db, 'users/{uid}/ngram_stats'))</code>
Corpus manifest fetch	<code>httpsCallable(functions, 'getCorpusManifest')</code>

08 Performance Budgets

METRIC	BUDGET
Input event → visual paint	< 5ms p95, < 12ms p99
Worker matrix update cycle	< 4ms per batch
Initial bundle (gzipped)	< 180KB for typing-critical path
Time to first interactive keystroke	< 1.2s on 4G / mid-tier laptop
Firestore sync flush	Non-blocking; failure invisible to UI

09 Security & Privacy

- Firestore Security Rules scope every read/write to `request.auth.uid == resource.data.uid` — no user can read another's performance data (full rules in Backend Schema doc).
- Raw keystroke *content* is never transmitted; only derived metrics (latency, error counts, WPM) leave the device.
- Cloud Functions enforce a write-rate ceiling per UID to prevent abuse of the sync endpoint.
- App Check (reCAPTCHA v3 / Play Integrity) attached to all callable functions to block non-app clients.
- All traffic over HTTPS/TLS 1.3 (default for Vercel's edge network & Firebase SDKs).

10 Testing Strategy

LAYER	APPROACH
Latency math	Unit tests on the EWMA and W(P) functions with fixed synthetic key-event fixtures — deterministic expected outputs.
Renderer	Visual regression (Playwright screenshot diffing) across the virtualized canvas buffer at varying viewport sizes.
Throughput	Synthetic keystroke replay at 2–3× realistic max human rate to validate the event bus doesn't drop events.
Sync merge logic	Property-based tests simulating concurrent multi-device writes to the same <code>pair_id</code> .
Security rules	Firebase Emulator Suite rules unit tests (allow/deny matrix per collection).
Cross-browser	Automated grid (Chrome, Firefox, Safari, Edge) validating <code>performance.now()</code> resolution assumptions hold.

11 Environments & CI/CD

ENVIRONMENT	FIREBASE PROJECT	VERCEL TARGET	DEPLOY TRIGGER
Local	Firebase Emulator Suite (Auth, Firestore, Functions)	<code>vercel dev</code>	Manual — <code>firebase emulators:start</code>
Preview	Shared staging project	Automatic PR preview URL	Every pull request → Vercel preview deployment
Staging	<code>kinetic-type-staging</code>	Staging environment alias	Merge to <code>main</code>
Production	<code>kinetic-type-prod</code>	Production domain	Tagged release, manual approval gate

GitHub Actions runs lint/test/build on every push; Vercel's native Git integration handles the actual preview/production promotion, so no separate deploy credentials are stored in CI.

12 Dependency Manifest

PACKAGE	PURPOSE	LICENSE
<code>react</code> , <code>react-dom</code>	Application shell & chrome UI	MIT
<code>dexie</code>	IndexedDB wrapper for local persistence	Apache-2.0
<code>zustand</code>	Lightweight client state	MIT
<code>firebase</code>	Auth, Firestore, Functions client SDK	Apache-2.0 (SDK) / managed backend
<code>comlink</code>	Ergonomic Web Worker message-passing wrapper	Apache-2.0
<code>posthog-js</code>	Self-hostable product analytics client	MIT
<code>@sentry/browser</code>	Error tracking, self-hostable backend	MIT SDK / FSL server ¹
<code>web-vitals</code>	Core Web Vitals measurement (Google)	Apache-2.0
<code>vite</code>	Build tooling & dev server	MIT
<code>vitest</code> / <code>playwright</code>	Unit & end-to-end / visual testing	MIT / Apache-2.0
<code>vercel</code>	CLI for local preview & deploy scripting (devDependency)	MIT (CLI) / managed platform

¹ See §02 footnote — self-hosted Sentry's server component ships under the Functional Source License; every other dependency above is OSI-approved open source.